**PROJECT REPORT**

# LOST AND FOUND PROPERTY MANAGEMENT SYSTEM

*Design and Development of a Database-Driven Web Application Using Django*

Submitted in partial fulfilment of the requirements for

Web Technologies and API Services

# Adventist University of Central Africa (AUCA)

## Group member:


| Student ID | Student Names |
|---|---|
| 20251MBI043 | NIYIBIZI Shaloom |
| 20251MBI041 | MUTETERI Charlotte |
| 20251MBI036 | KWIZERA Marie Francine |


**Lecturer**: Kundan Kumar

**Submission Date**: 18 / 10 / 2026

# Table of Contents

Declaration on the use of AI tools ..... 4
Abstract ..... 5
Chapter 1: Introduction ..... 6
1.1 Background and problem statement ..... 6
1.2 Objectives ..... 6
1.3 Scope ..... 6
1.4 Users and roles ..... 6
1.5 Main features ..... 7
Chapter 2: Requirements Analysis and Methodology ..... 7
2.1 Sources of requirements ..... 7
2.2 Functional requirements ..... 7
2.3 Non-functional requirements ..... 8
2.4 Development methodology ..... 8
2.5 Tools and technologies ..... 9
Chapter 3: System Design ..... 9
3.1 Architecture ..... 9
3.2 Database design ..... 10
3.3 Relationships and cardinality ..... 11
3.4 Claim lifecycle ..... 13
3.5 User interface design ..... 13
Chapter 4: Implementation ..... 14
4.1 Project structure and configuration ..... 14
4.2 Requirements implemented ..... 15
4.3 Matching engine ..... 16
4.4 Claim workflow ..... 16
4.5 Challenges and solutions ..... 16
4.6 Installation and running ..... 17
Chapter 5: Testing and Results ..... 18
5.1 Testing approach ..... 18
5.2 Automated test suite ..... 18
5.3 Functional test cases ..... 19

5.4 Results ..... 20
5.5 Screenshots ..... 21
Chapter 6: Conclusion and Recommendations ..... 30
6.1 Conclusion ..... 30
6.2 Limitations ..... 31
6.3 Recommendations ..... 31
References ..... 32
Appendix A: Submission checklist ..... 32
Appendix B: Suggested demonstration sequence ..... 32

# DECLARATION ON THE USE OF AI TOOLS

The course brief allows AI-assisted tools, provided students can explain the code they submit. We used Claude, an AI assistant made by Anthropic, for three tasks:

- Reviewing the source code against the project brief, which found one defect (section 5.4) and several small clean-ups;
- Making those fixes, with an automated test for each change in behavior;
- Drafting this report from the source code.

We checked every change by running the full test suite (158 tests, all passing) and checked the report against the code. Each group member has studied the parts they present and can explain them.

# ABSTRACT

On a university campus, lost property is usually reported by word of mouth, and found items wait at the security office with no reliable way to reach their owners. This project presents LOSTFinder, a database-driven web application built with Python 3.14 and Django 6.1. It gives students, staff and campus security one place to report, match and return lost items.

Users report lost or found items with a photo, category, location and date. A matching engine scores each new report against reports of the opposite type and suggests likely pairs. An owner then submits a claim with private proof of ownership. A security officer approves or rejects the claim and records the handover.

The system has four Django apps and nine models, an AdminLTE dashboard with live statistics, and role-based access to pages. All 158 automated tests pass. This report covers the requirements, design, implementation and testing, and recommends further work.

# CHAPTER 1: INTRODUCTION

## 1.1 Background and problem statement

On most campuses a lost item is reported by word of mouth, and found items wait at the security office with no way to reach their owner. This causes four problems:

- There is no central record of what has been lost or found.
- Owners and finders cannot find each other, so items stay unclaimed.
- There is no check that the person collecting an item really owns it.
- There is no record of who handed over an item, to whom, and when.

LOSTFinder gives both sides one place to report items. It also gives security a record of who proved ownership and who collected each item.

## 1.2 Objectives

The main objective is to build a web application that helps the campus community recover lost items quickly and safely. The specific objectives are to:

1. Let users report lost and found items with a photo, category, location and date.
2. Suggest likely matches between lost and found reports automatically.
3. Let security officers verify ownership claims and record each handover, with a full audit log.
4. Give each role a dashboard with summary statistics, charts and recent records.
5. Restrict pages to signed-in users, and review pages to security officers and administrators.
6. Meet the course brief: several Django apps, related models, CRUD with validation, and an AdminLTE interface.

## 1.3 Scope

In scope: a web application for one campus, with four user roles, in-app notifications, and a SQLite or PostgreSQL database.

Out of scope: email or SMS notifications, a mobile app, several campuses, and deployment to a public server.

## 1.4 Users and roles

| Role | How a user gets it | What they can do |
|---|---|---|
| Student | Default for everyone who registers | Report lost and found items, review match suggestions, submit and cancel claims |
| University staff | Set by an administrator | Same as a student |
| Security officer | Set by an administrator in the | Everything above, plus review claims and |

| Role | How a user gets it | What they can do |
|---|---|---|
| | Django admin | record handovers |
| Administrator | Django is_staff flag (for example, the superuser) | Everything above, plus the Django admin site and user statistics |

### 1.5 Main features

- **Lost and found reports** with a photo (up to 5 MB), category, location and date, plus search and filters by category and status.
- **Automatic matching.** Each saved report is scored against reports of the opposite type, out of 100. Pairs scoring 35 or more are suggested to the owner.
- **Claims workflow.** A claim moves from submitted to under review, then approved or rejected, and is completed at handover. Every step is logged and notifies the people involved.
- **Role-based dashboard** built on AdminLTE, with summary cards, charts and tables that refresh every 30 seconds.
- **Accounts** with email sign-in and a required profile (onboarding) before first use.
- **In-app notifications** with an unread count in the top bar.

# CHAPTER 2: REQUIREMENTS ANALYSIS AND METHODOLOGY

### 2.1 Sources of requirements

The requirements came from two sources: the course project brief, and the lost-and-found process the application supports. That process is to report an item, find a likely match, prove ownership, and hand the item over.

### 2.2 Functional requirements

| ID | Requirement | Who |
|---|---|---|
| FR1 | Register and sign in with a university email, then complete a profile before first use. | All users |
| FR2 | Create, view, edit and delete lost item reports. Only the reporter can edit or delete a report. | Students and staff |
| FR3 | Create, view, edit and delete found item reports, with the same ownership rule. | Students and staff |
| FR4 | Search reports by keyword and filter them by category and status. | All users |

| ID | Requirement | Who |
|---|---|---|
| FR5 | Suggest matches automatically when a report is saved, and let the owner accept or reject each one. | Students and staff |
| FR6 | Submit a claim with proof of ownership, and cancel it while it is still open. | Students and staff |
| FR7 | Review claims: mark under review, approve or reject, with comments. | Security officers, administrators |
| FR8 | Record the handover of an approved claim. | Security officers, administrators |
| FR9 | Notify users when their claims or items change. | All users |
| FR10 | Show a role-based dashboard with summary cards, charts and tables. | All users |
| FR11 | Manage all records and user roles in the Django admin. | Administrators |

## 2.3 Non-functional requirements

- **Security.** Every app page requires sign-in, and review pages require the reviewer role. Forms are protected against cross-site request forgery. Passwords are hashed and must pass Django's password validators. The secret key and database credentials are kept in a `.env` file outside version control.
- **Privacy.** Claim evidence is visible only to the claimant and reviewers.
- **Data integrity.** Database constraints, transactions and protected deletes keep related records consistent.
- **Usability.** Every page shares one layout, shows errors next to the field they concern, and supports light and dark themes.
- **Responsiveness.** The Bootstrap 5 grid adapts each page to phone and desktop screens.
- **Maintainability.** The code is split into four apps with separate responsibilities, and 158 automated tests check its behaviour.
- **Portability.** All front-end libraries are stored in the project, so the app works without internet access, on SQLite or PostgreSQL.

## 2.4 Development methodology

The project followed an iterative, incremental approach. Each iteration added one area of the system, with tests, then refined it:

1. Analyse the brief and list the requirements above.
2. Design the data model and ER diagram.
3. Build the apps in order: `accounts`, `items`, `claims`, then `dashboard`.

4. Integrate AdminLTE, first in the app pages and then in the Django admin.
5. Write automated tests with each feature, and fix what they find.

The code is versioned with Git and hosted on GitHub. The history records 8 commits from 13 to 16 October 2026, each adding or refining one area, such as the dashboard (14 October) and the AdminLTE admin theme (16 October).

### 2.5 Tools and technologies

| Purpose | Tool |
|---|---|
| Language | Python 3.14.7 |
| Web framework | Django 6.1.2 |
| Database | SQLite by default; PostgreSQL through psycopg2-binary 2.9.13 |
| Dashboard template | AdminLTE 4.9.1, with Bootstrap 5.3.8 |
| Icons and charts | Bootstrap Icons 1.13.1, Chart.js 4.5.1 |
| Image uploads | Pillow 12.3.0 |
| Configuration | python-dotenv 1.2.4, reading a .env file |
| Dependencies | uv, with versions locked in uv.lock and requirements.txt |
| Template linting | djLint 1.46.4 |
| Version control | Git and GitHub |
| Editor | Visual Studio Code |

# CHAPTER 3: SYSTEM DESIGN

### 3.1 Architecture

LOSTFinder follows Django's Model–View–Template pattern: models define the database tables, views handle each request, and templates render the HTML. The project, config, is split into four Django apps with 30 function-based views between them. Each app has its own urls.py, and config/urls.py includes them under a prefix.

| App | URL prefix | Responsibility | Models | Views |
|---|---|---|---|---|
| accounts | /accounts/ | Registration, email sign-in, onboarding, profile, role checks | UserProfile | 5 |
| items | /items/ | Lost and found reports, the matching engine, match review | ItemCategory, LostItem, FoundItem, | 12 |

| App | URL prefix | Responsibility | Models | Views |
|---|---|---|---|---|
| | | | MatchSuggestion | |
| claims | /claims/ | Claims, the review queue, handovers, notifications | Claim, ClaimReviewLog, ItemHandover, Notification | 10 |
| dashboard | / | Public home page, role-based dashboard, live metrics (JSON) | None (reads the other apps) | 3 |

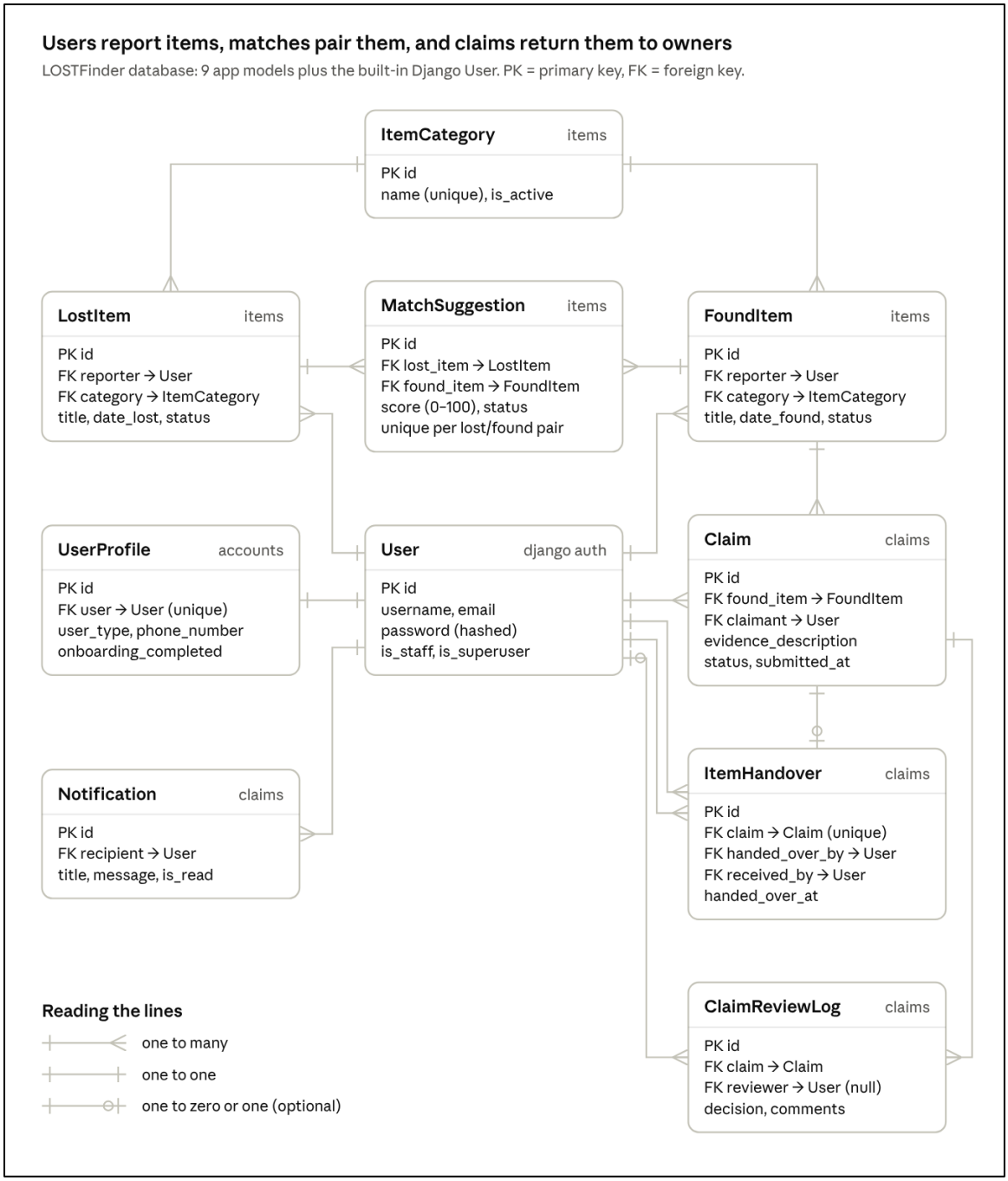
The request for a lost item's page, /items/lost/5/, shows how a URL reaches a view and becomes a response:

1. config/urls.py matches the items/ prefix and hands the rest of the path to items/urls.py through include().
2. items/urls.py matches lost/<int:pk>/, converts 5 to an integer, and calls lost_item_detail(request, pk=5).
3. @login_required sends a signed-out visitor to the login page. @profile_completed_required sends a user without a completed profile to onboarding.
4. The view loads the item with get_object_or_404, so an unknown id returns a 404 page.
5. render() fills items/item_detail.html with the item. That template extends base.html, which adds the AdminLTE top bar, sidebar and footer.
6. Django returns the finished HTML to the browser as an HttpResponse.

Pages link to each other with named URLs, for example {% url 'lost_item_detail' item.pk %}, so a path can change without breaking links.

## 3.2 Database design

The database has 10 tables: Django's built-in User and 9 models in the three apps that store data. User is the hub, linked to 7 of the 9 models.




LOSTFinder ER diagram · 10 tables, 15 relationships

## 3.3 Relationships and cardinality

The design uses 13 ForeignKeys and 2 OneToOneFields, and one many-to-many link through an explicit model. `User` is Django's built-in `auth.User`.

| Relationship | Field and type | Cardinality | On delete |
|---|---|---|---|

| Relationship | Field and type | Cardinality | On delete |
|---|---|---|---|
| User – UserProfile | UserProfile.user, OneToOneField | 1 : 1. Each user has exactly one profile. | CASCADE |
| User – LostItem | LostItem.reporter, ForeignKey | 1 : N. A user reports many lost items; each has one reporter. | PROTECT |
| User – FoundItem | FoundItem.reporter, ForeignKey | 1 : N | PROTECT |
| ItemCategory – LostItem | LostItem.category, ForeignKey | 1 : N. A category holds many reports; each report has one category. | PROTECT |
| ItemCategory – FoundItem | FoundItem.category, ForeignKey | 1 : N | PROTECT |
| LostItem – FoundItem | Through MatchSuggestion (lost_item and found_item ForeignKeys) | M : N. A lost item can match many found items and the reverse; each pair appears once. | CASCADE |
| FoundItem – Claim | Claim.found_item, ForeignKey | 1 : N. Several people may claim one found item. | PROTECT |
| User – Claim | Claim.claimant, ForeignKey | 1 : N. One claim per user per item. | PROTECT |
| Claim – ClaimReviewLog | ClaimReviewLog.claim, ForeignKey | 1 : N. Every status change adds a log row. | CASCADE |
| User – ClaimReviewLog | ClaimReviewLog.reviewer, ForeignKey (nullable) | 0..1 : N. Empty for steps the system or claimant takes. | SET_NULL |
| Claim – ItemHandover | ItemHandover.claim, OneToOneField | 1 : 0..1. A claim has at most one handover. | PROTECT |
| User – ItemHandover | handed_over_by and received_by, two ForeignKeys | 1 : N each | PROTECT |
| User – Notification | Notification.recipient, ForeignKey | 1 : N | CASCADE |

The on-delete rules follow from what each record means:

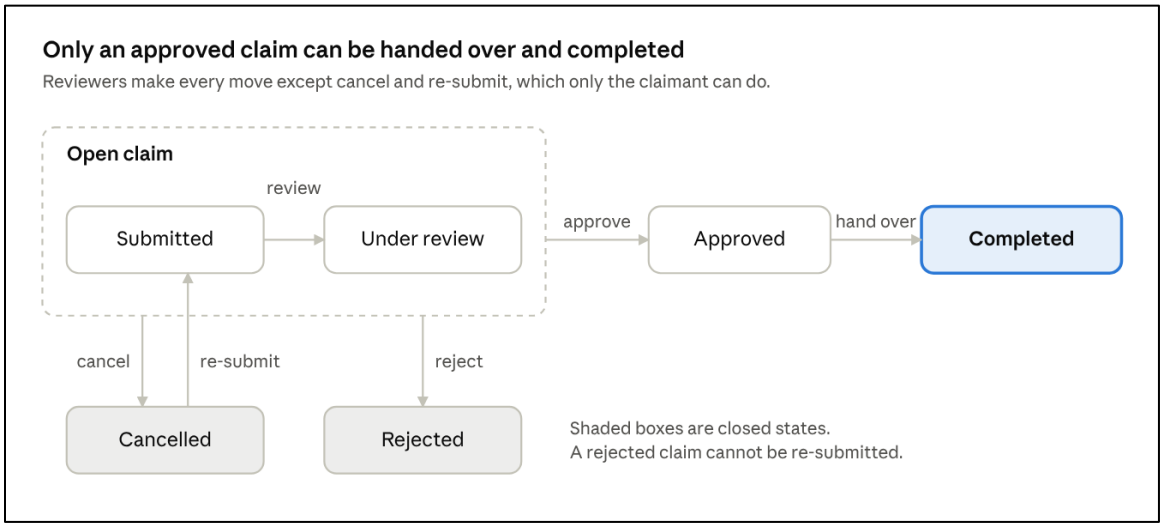
- **PROTECT** keeps history safe. A category or user that has reports cannot be deleted, and a found item with claims cannot be deleted.
- **CASCADE** removes records that only make sense with their parent, such as match suggestions and notifications.
- **SET_NULL** keeps the review log as an audit trail even if a reviewer's account is removed.

The database also enforces these rules:

- A lost and found pair can have only one match suggestion, and its score must be 0 to 100 (a unique constraint and a check constraint).
- A user can claim a given found item only once (a unique constraint).
- In model validation (`clean()`), a user cannot claim an item they found, a handover needs an approved claim, and only the claimant can receive the item.

## 3.4 Claim lifecycle

A claim always ends in one of three closed states: completed, rejected or cancelled. All changes go through `claims/services.py`, which also keeps the found item's status in step.




Claim lifecycle · 6 statuses

Approving a claim also rejects every other open claim on the same item, and notifies those claimants.

## 3.5 User interface design

All signed-in pages extend `base.html`, which uses the AdminLTE layout: a top bar, a sidebar, a page header with breadcrumbs, the content area and a footer. Visitors see `base_public.html`, which has a top bar only.

The sidebar groups links by task and shows only what the user's role allows:

| Sidebar section | Links | Shown to |
|---|---|---|
| Dashboard | Dashboard | Everyone |
| Reports | Lost items, Found items, Report lost item, Turn in found item | Everyone |
| Recovery | My matches, My claims, Notifications (with unread counts) | Everyone |
| Security desk | Review queue, Awaiting handover | Security officers and administrators |
| Administration | Django admin | Administrators |
| Account | My profile | Everyone |

The dashboard also depends on the role. Students and staff see their own open reports, matches to review and claims. Security officers see claims to review, items awaiting handover, items in custody and recent returns. A theme menu in the top bar switches between light, dark and automatic modes.

# CHAPTER 4: IMPLEMENTATION

## 4.1 Project structure and configuration

The project is organised by app, with shared templates and static files at the root. Every app also has its own `migrations/` folder.

```
LOSTFinder/
├── config/        settings.py, urls.py, wsgi.py, asgi.py
├── accounts/      models, forms, views, urls, admin, backends.py, decorators.py
├── items/         models, forms, views, urls, admin, matching.py
│   └── management/commands/   seed_demo.py, run_matching.py
├── claims/        models, forms, views, urls, admin, services.py, context_processors.py
├── dashboard/     views, urls, metrics.py, templatetags/
├── templates/     base.html, base_public.html, layouts/, admin/, one folder per app
├── static/        css/, js/, image/, vendor/ (AdminLTE, Bootstrap, Bootstrap Icons, Chart.js)
├── media/         uploaded photos, created at run time
├── manage.py
├── .env.example
└── pyproject.toml, uv.lock, requirements.txt
```

The main settings in `config/settings.py` are:

- `INSTALLED_APPS` lists the four project apps after Django's own.
- `TEMPLATES` reads `templates/`, and a context processor (`claims_nav`) adds the sidebar's badge counts to every page.
- `STATICFILES_DIRS` points to `static/`, and `MEDIA_ROOT` to `media/` for uploads.

- STATIC_ROOT points to staticfiles/, where collectstatic gathers static files for deployment.
- AUTHENTICATION_BACKENDS puts the email backend before Django's default one.
- LOGIN_URL and LOGIN_REDIRECT_URL send users to the login page and then to the dashboard.
- SECRET_KEY, DEBUG and the database settings are read from .env with python-dotenv.

## 4.2 Requirements implemented

Each requirement in the project brief maps to specific code, listed below.

| Brief requirement | How LOSTFinder meets it | Where |
|---|---|---|
| URL routing and views | Project URLs include each app's URLs. 30 function-based views, with dynamic parameters such as <int:pk> and <int:found_pk>. | config/urls.py, each app's urls.py and views.py |
| Templates and inheritance | base.html (signed-in layout) and base_public.html (visitor layout). All 19 pages extend one of them. Repeated parts are included partials: header, sidebar, cards, tables. | templates/ |
| CRUD | Lost and found items: create, list with search, filters and pages of 12, detail, edit, and delete with a confirmation page. Claims: create, view, cancel, review and hand over. | items/views.py, claims/views.py |
| Forms and validation | ModelForms for items, profiles and handovers; plain Forms for claims and reviews. Rules: dates cannot be in the future, images are at most 5 MB, phone numbers have 10 digits, emails are unique, passwords pass Django's validators, claim evidence is at least 20 characters. Report forms offer only active categories. | items/forms.py, accounts/forms.py, claims/forms.py |
| AdminLTE dashboard | AdminLTE 4 with Bootstrap 5.3, Bootstrap Icons and Chart.js, stored in static/vendor/ and loaded with {% static %}. Sidebar and top bar on every signed-in page. The dashboard has 4 summary cards, 3 smaller stat cards, 3 charts and tables of database records. It refreshes every 30 seconds from /dashboard/metrics/. | templates/base.html, templates/layouts/, templates/dashboard/, dashboard/metrics.py |
| Django admin | All 9 models registered, with list columns, search and filters. A claim's review log and | each app's admin.py, |

| Brief requirement | How LOSTFinder meets it | Where |
|---|---|---|
| | handover are edited inline on the claim page. The admin site is restyled with AdminLTE to match the app, and saving a report there re-runs matching. | templates/admin/ |
| Authentication | Registration, email sign-in through a custom backend, and logout by POST only. @login_required protects every app page, and @reviewer_required returns 403 to anyone who is not a reviewer. | accounts/ |

Only the person who reported an item can edit or delete it. The edit and delete views look the item up with reporter=request.user, so anyone else gets a 404.

### 4.3 Matching engine

Saving a lost or found report scores it against every open report of the opposite type (items/matching.py). Text similarity combines shared words (60%) with character-level similarity from Python's difflib (40%). Pairs scoring 35 or more are saved as suggestions, with the reasons shown to the user.

| Signal | Points |
|---|---|
| Same category | 30 |
| Title similarity | up to 25 |
| Location similarity | up to 20 |
| Description similarity | up to 15 |
| Date proximity (same day 10, down to 1 within 14 days) | up to 10 |

### 4.4 Claim workflow

All status changes go through claims/services.py inside database transactions, following the lifecycle in section 3.4. Each change writes a review-log row, notifies the claimant, finder or reviewers, and updates the found item's status: Available, Claim pending, Claim approved, then Returned to owner.

### 4.5 Challenges and solutions

The hardest parts were keeping related records consistent and fitting Django's defaults to a campus workflow.

| Challenge | How it was solved |
|---|---|
| Django signs users in by username, but students know their university email. | A custom authentication backend (accounts/backends.py) looks users up by email, ignoring case. Registration builds a unique username from the email behind the scenes. |
| A found item's status depends on its claims, and a lost item's status depends on its matches. Updating them by hand in each view would drift. | Statuses are derived from related records in one function per model, called after every change inside transaction.atomic, so a failed step rolls back everything. |
| Re-scoring matches after an edit could erase a user's accept or reject decision. | Only pending suggestions are re-scored. Accepted and rejected ones are kept, and a user's own lost and found reports are never matched. |
| Deleting a found item that has claims would destroy the audit trail. | PROTECT blocks the delete. The view catches ProtectedError and shows a clear message instead of an error page. |
| Claim evidence is private, but the finder should still see progress. | The claim page checks the viewer's role. Reviewers and the claimant see the evidence; the finder sees only the status; anyone else gets a 404. |
| The Django admin has its own layout, which clashed with the AdminLTE app. | templates/admin/base.html keeps all of Django's template blocks but wraps them in the AdminLTE layout. static/css/admin.css maps the admin's colours to the light and dark themes. |
| Browsers kept old CSS after changes. | A custom {% asset %} template tag adds the file's last-modified time to each URL, so a changed file is always downloaded again. |

## 4.6 Installation and running

The project needs Python 3.14. With the included db.sqlite3 and media/ folder, skip steps 3 to 5.

1. Install the dependencies: uv sync, or pip install -r requirements.txt.
2. Copy .env.example to .env. It already contains working values for local use.
3. Create the database tables: python manage.py migrate.
4. Load the demo data: python manage.py seed_demo. This adds 8 categories, 6 lost and 6 found items, match suggestions, claims at every stage, and the four demo accounts below.
5. Create an administrator: python manage.py createsuperuser. Sign in to the app with its email, not the username.
6. Start the server with python manage.py runserver and open http://127.0.0.1:8000/. The Django admin is at http://127.0.0.1:8000/admin/.

7. Optionally, run the 158 automated tests with `python manage.py test`.

These accounts are ready for marking:

| Account | Email | Password | Role |
|---|---|---|---|
| Superuser | admin@gmail.com | Admin@123 | Administrator, with Django admin access |
| Demo | officer@lostfinder.test | Demo-Pass-2026! | Security officer |
| Demo | amina@lostfinder.test | Demo-Pass-2026! | Student |
| Demo | daniel@lostfinder.test | Demo-Pass-2026! | Student |
| Demo | grace@lostfinder.test | Demo-Pass-2026! | University staff |

# CHAPTER 5: TESTING AND RESULTS

## 5.1 Testing approach

The system was tested with Django's built-in test framework. Each run creates a temporary test database, so real data is never touched. Most tests use Django's test client to sign in as a student, staff member, security officer or administrator. They request pages as that user and check the response, the redirect and the database afterwards. The matching engine and claim services are also tested directly as units.

## 5.2 Automated test suite

| Test file | What it checks | Tests |
|---|---|---|
| accounts/tests.py | Registration, email sign-in, the onboarding gate, roles, sign-in pop-ups, profile form validation | 31 |
| dashboard/tests.py | Home page, role-based dashboards, live metrics and charts, shared layout, bundled assets with no CDN | 27 |
| claims/tests.py | Claim workflow rules and the permissions on each claim page | 24 |
| items/tests.py | Matching scores, matching on save in the app and the admin, accepting and rejecting matches, edit and delete permissions | 24 |
| claims/test_pages.py | Match, claim, notification and review-queue pages | 20 |
| items/test_forms.py | Report forms, photo upload and the size limit, date errors, active categories only | 9 |

| Test file | What it checks | Tests |
|---|---|---|
| items/test_lists.py | Lost and found lists: filters, status counts, paging | 8 |
| dashboard/test_admin.py | The AdminLTE-styled Django admin, admin login and access | 8 |
| items/test_seed.py | The demo data command | 7 |
| **Total** | | **158** |

### 5.3 Functional test cases

Each case below tests one requirement from section 2.2 and is checked by the automated test named beside it.

| ID | Test case | Expected result | Automated test | Result |
|---|---|---|---|---|
| TC01 | Register with an email already in use (FR1) | Form shown again; no account created | test_duplicate_email_rejected | Pass |
| TC02 | Sign in with a wrong password (FR1) | Error message; user stays signed out | test_wrong_password_rejected | Pass |
| TC03 | Open the app before completing the profile (FR1) | Redirected to onboarding | test_incomplete_profile_is_sent_to_onboarding | Pass |
| TC04 | Report an item with a future date (FR2) | Error shown next to the date field | test_future_date_rejected_with_message | Pass |
| TC05 | Upload a 6 MB photo (FR2) | "The image must not exceed 5 MB."; nothing saved | test_oversized_photo_rejected | Pass |
| TC06 | Edit or delete another user's report (FR2, FR3) | 404 page; report unchanged | test_non_owner_cannot_edit_or_delete | Pass |
| TC07 | Delete a found item that has a claim (FR3) | Item is not deleted | test_found_item_with_claims_cannot_be_deleted | Pass |
| TC08 | Save a lost report similar to a found one (FR5) | Suggestion created; lost item marked as matched | test_match_lost_item_creates_suggestion_and_flags_matched | Pass |
| TC09 | Save lost and found reports by the same user (FR5) | No suggestion | test_own_reports_are_not_matched | Pass |

| ID | Test case | Expected result | Automated test | Result |
|---|---|---|---|---|
| TC10 | Claim an item you found yourself (FR6) | Claim refused | test_finder_cannot_claim_own_item | Pass |
| TC11 | Submit claim evidence under 20 characters (FR6) | Form error | test_short_evidence_is_rejected | Pass |
| TC12 | A student opens the review queue (FR7) | 403 Forbidden | test_review_queue_is_reviewers_only | Pass |
| TC13 | Approve one of two claims on an item (FR7) | The other claim is rejected and its claimant notified | test_approve_rejects_competing_claims | Pass |
| TC14 | Record a handover for an unapproved claim (FR8) | Refused; no handover saved | test_handover_requires_approval | Pass |
| TC15 | Record a handover for an approved claim (FR8) | Claim completed; found and lost items marked returned | test_handover_completes_claim_and_returns_items | Pass |
| TC16 | Open the dashboard as a security officer (FR10) | Security Desk Dashboard with the claim queue; no Administration menu | test_security_officer_dashboard | Pass |
| TC17 | A student opens /admin/ (FR11) | Redirected to the admin login | test_non_staff_are_sent_to_admin_login | Pass |
| TC18 | Edit your profile as a security officer (FR1) | Changes saved; role unchanged | test_profile_edits_are_saved_and_keep_the_role | Pass |
| TC19 | An administrator adds a lost report in the Django admin (FR11) | Match suggestions created, as in the app | test_admin_save_generates_suggestions | Pass |

## 5.4 Results

On 16 October 2026, all 158 tests passed in 260 seconds. Django's system check reported no issues, and makemigrations --check found no model changes missing from the migrations.

A defect found during review has been fixed. The **My profile** page showed a success message but did not save changes, because profile_view called form.save(commit=False) without saving the result. It now calls form.save(), and test TC18 checks that the changes are stored and the user's role is unchanged. In the same review, the Django admin was changed to re-run matching when a report is saved (TC19).

## 5.5 Screenshots

The figures below show the main functions of the system.

***Figure 5.1:*** *Public home page*

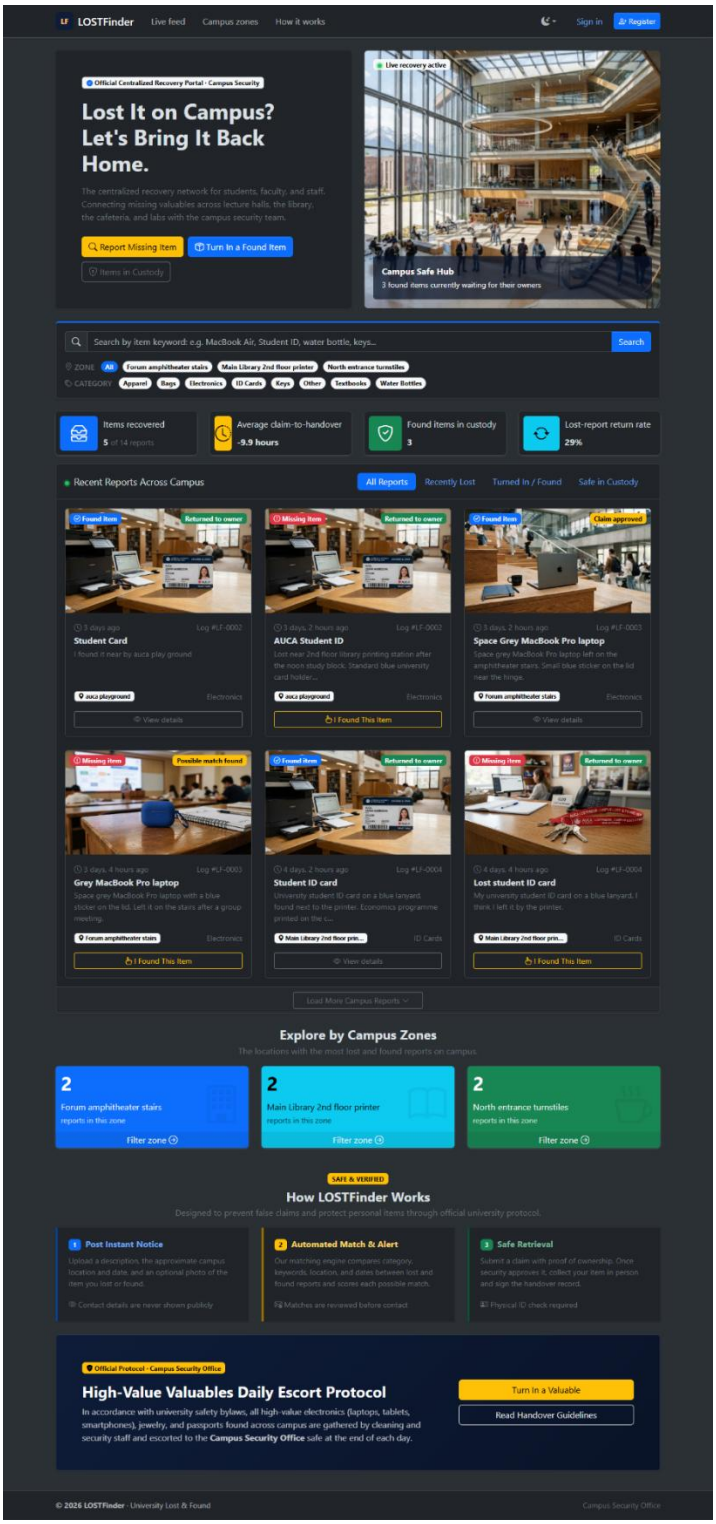


***Figure 5.2:*** *Student dashboard with summary cards, charts and tables*

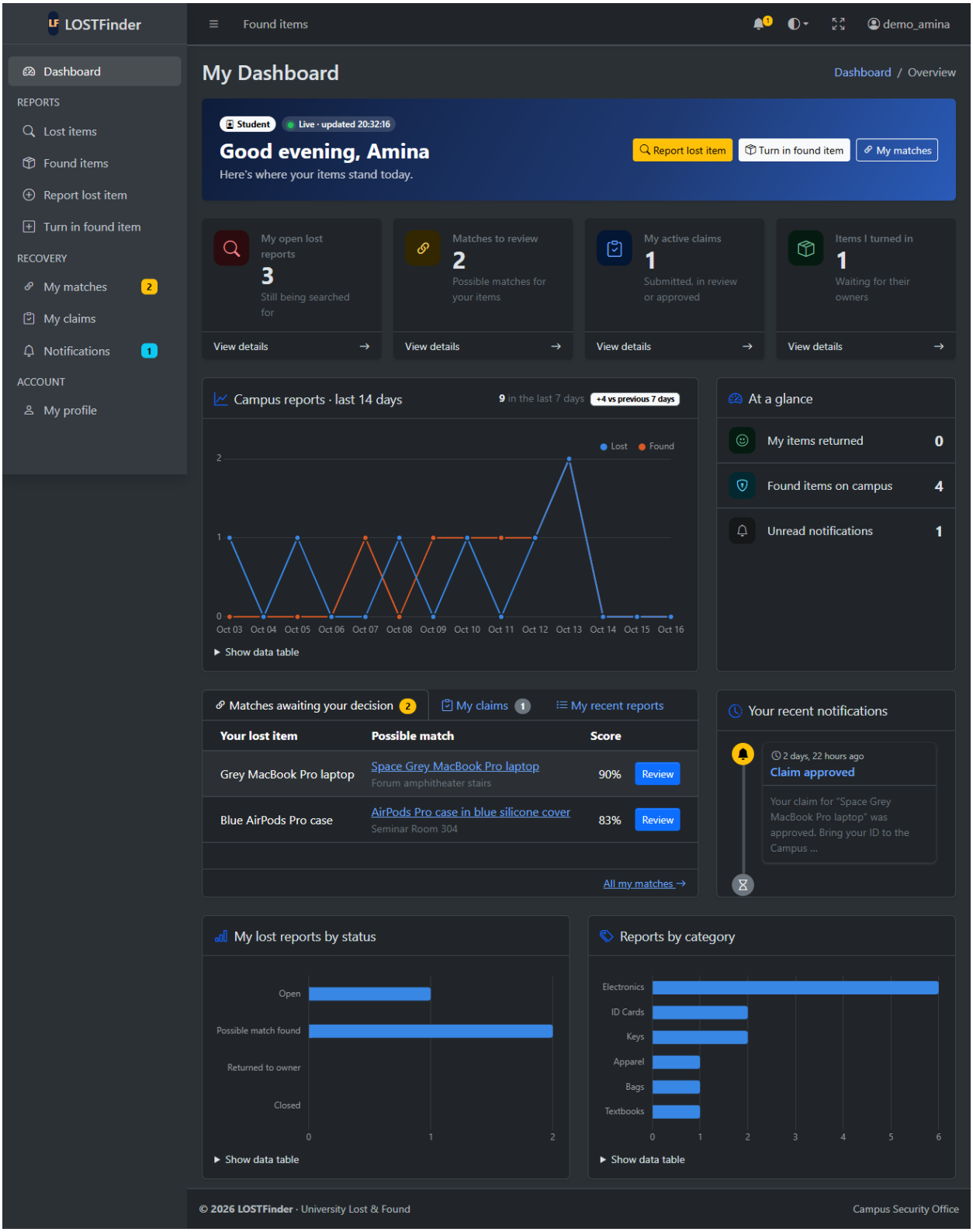


***Figure 5.3:*** *Security desk dashboard*

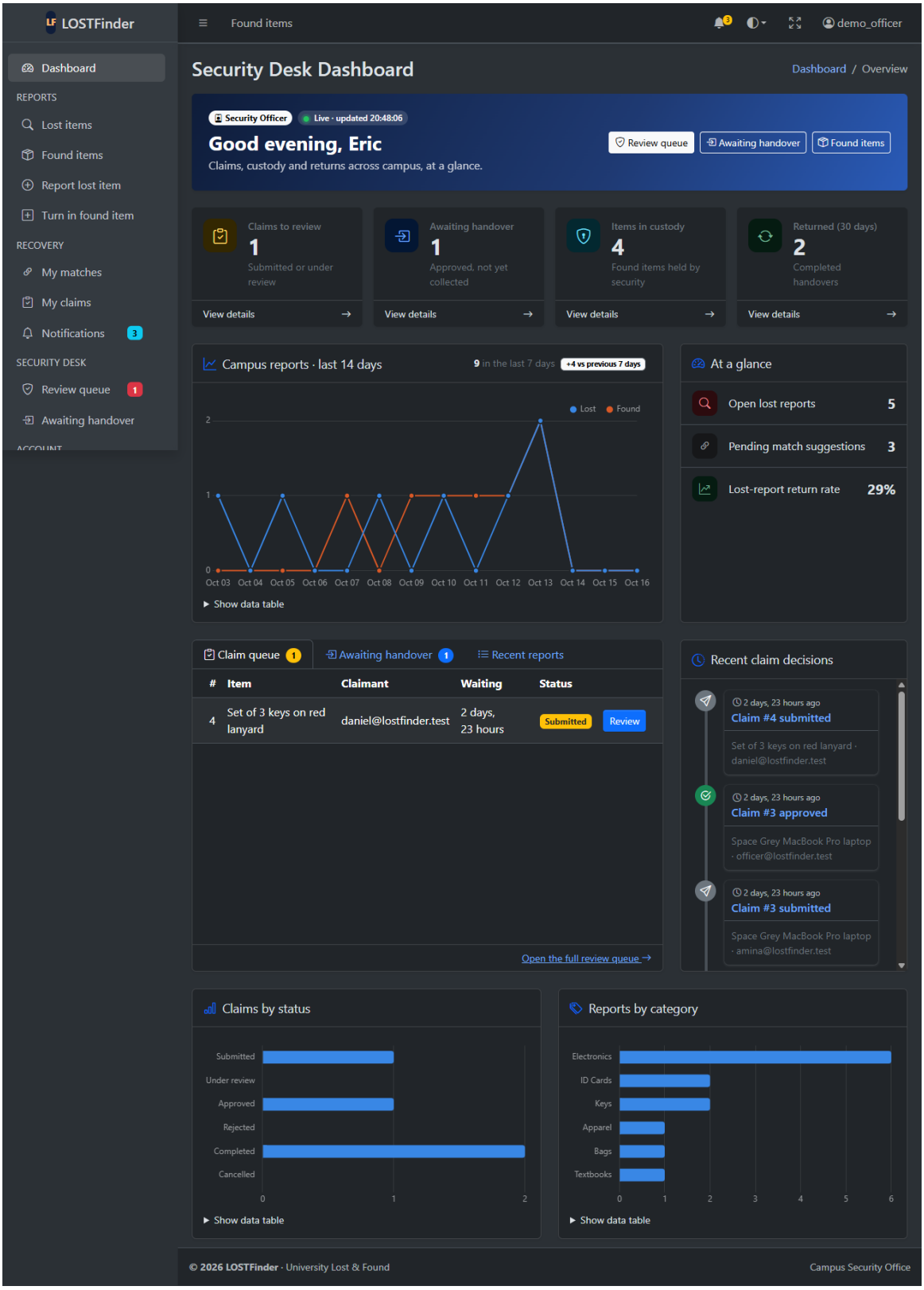


***Figure 5.4:*** *Lost item form with a validation error*

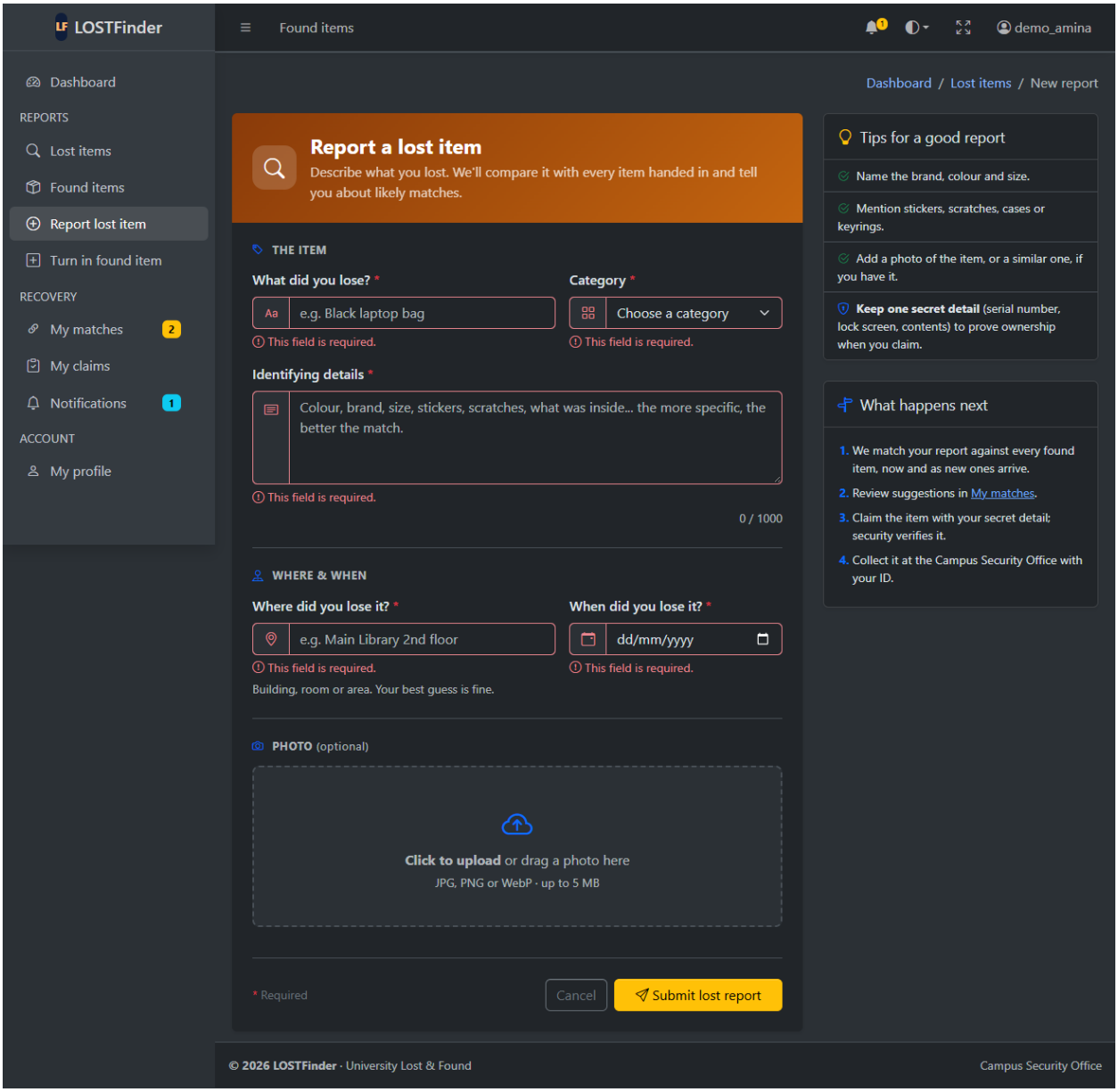


***Figure 5.5:*** *Lost items list with search and filters*

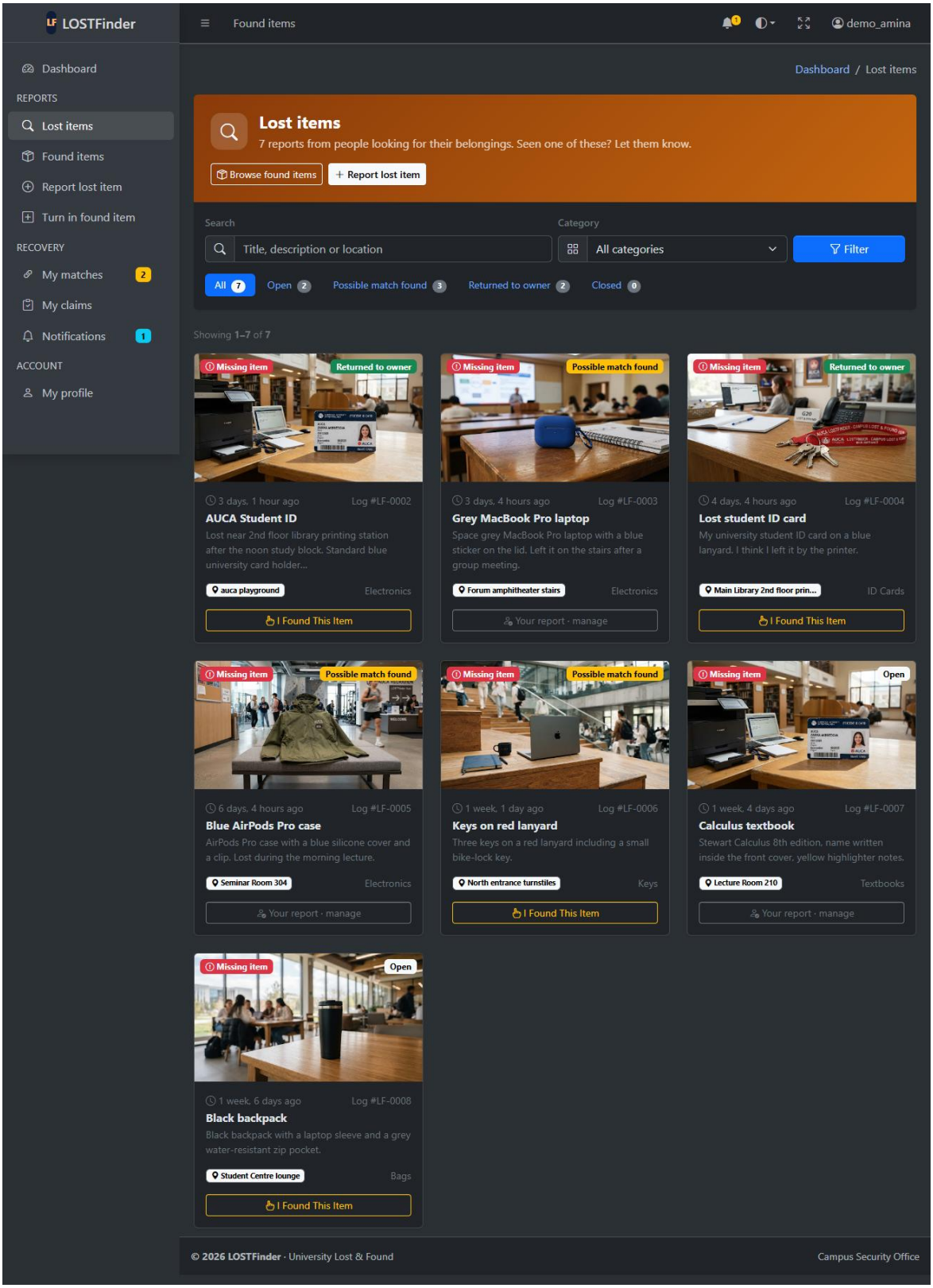


***Figure 5.6:*** *Delete confirmation page*

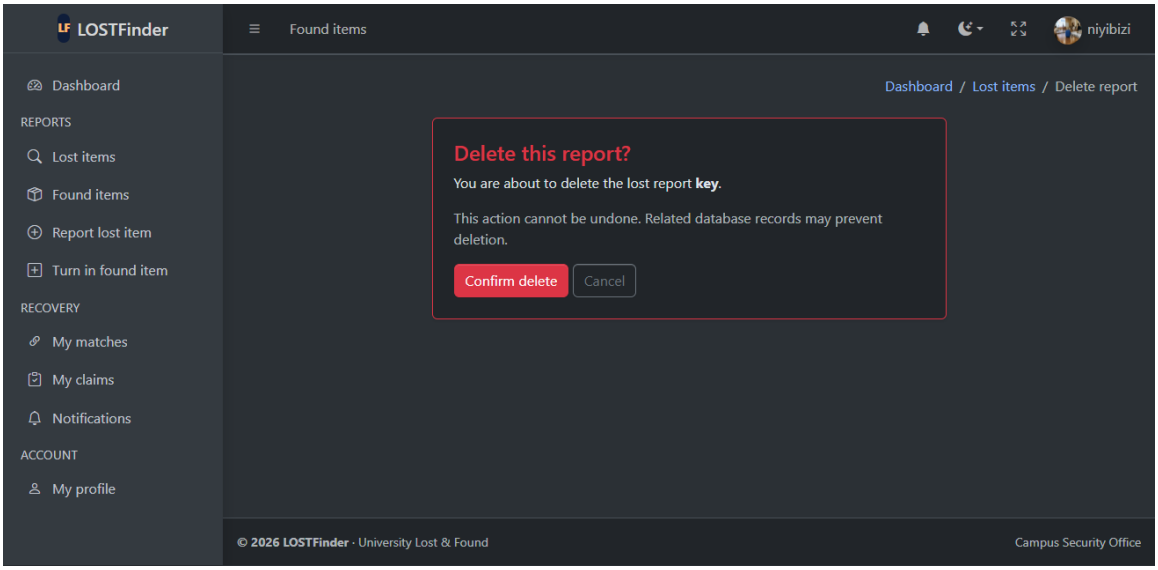


***Figure 5.7:*** *Match suggestions*

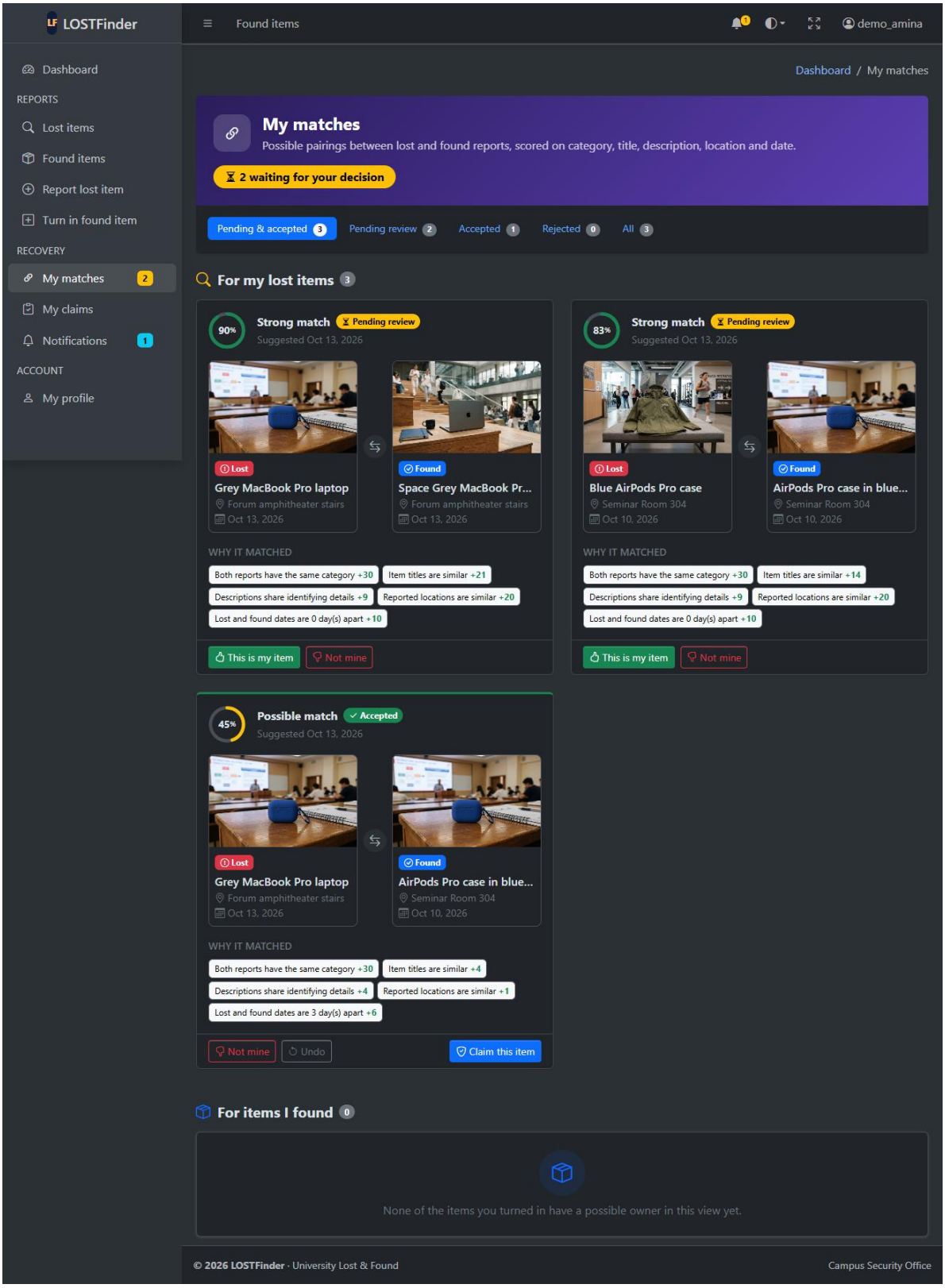


***Figure 5.8:*** *My claims page, with an approved claim ready to collect*

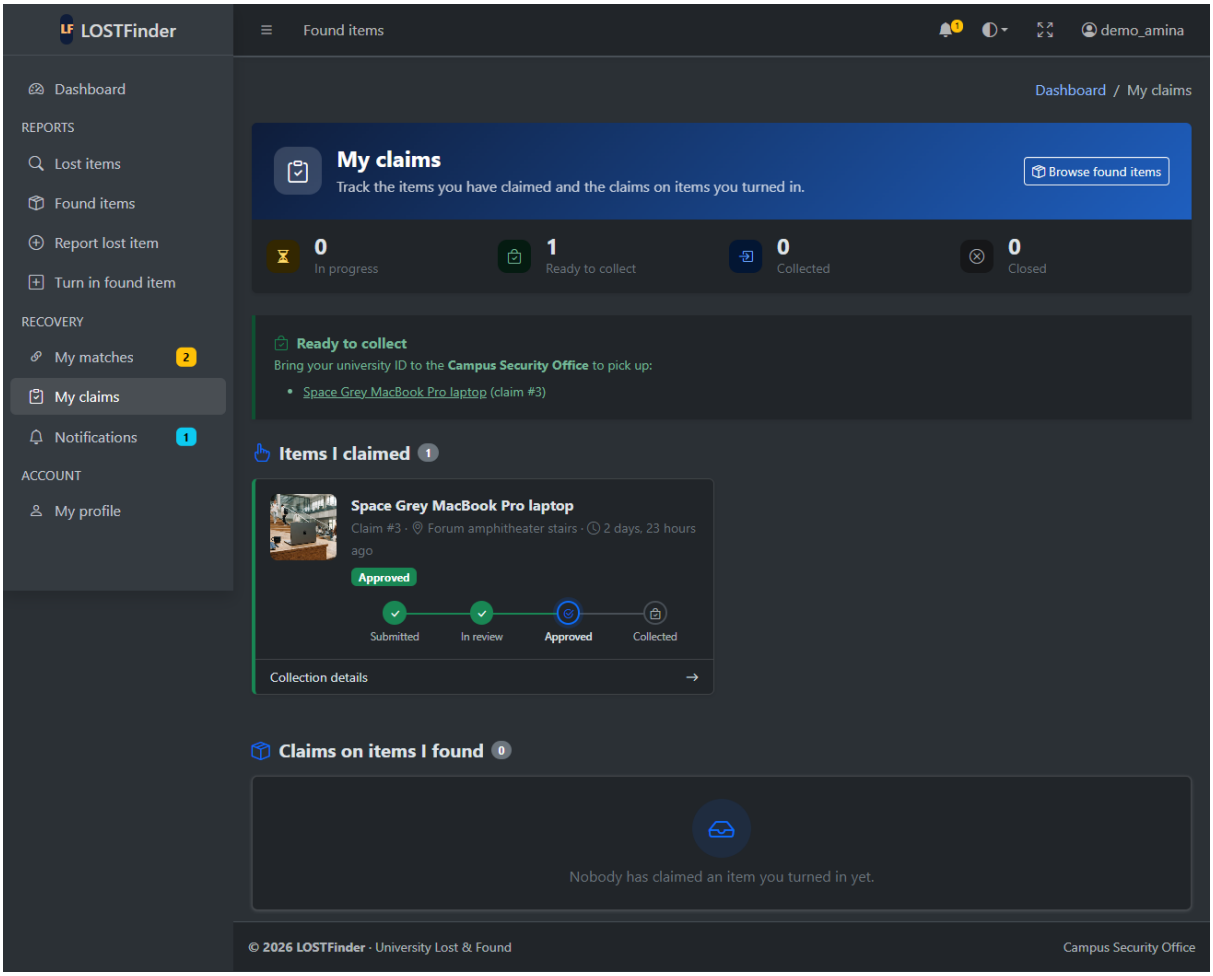


***Figure 5.9:*** *Security desk dashboard for an administrator, with the registered-users count*

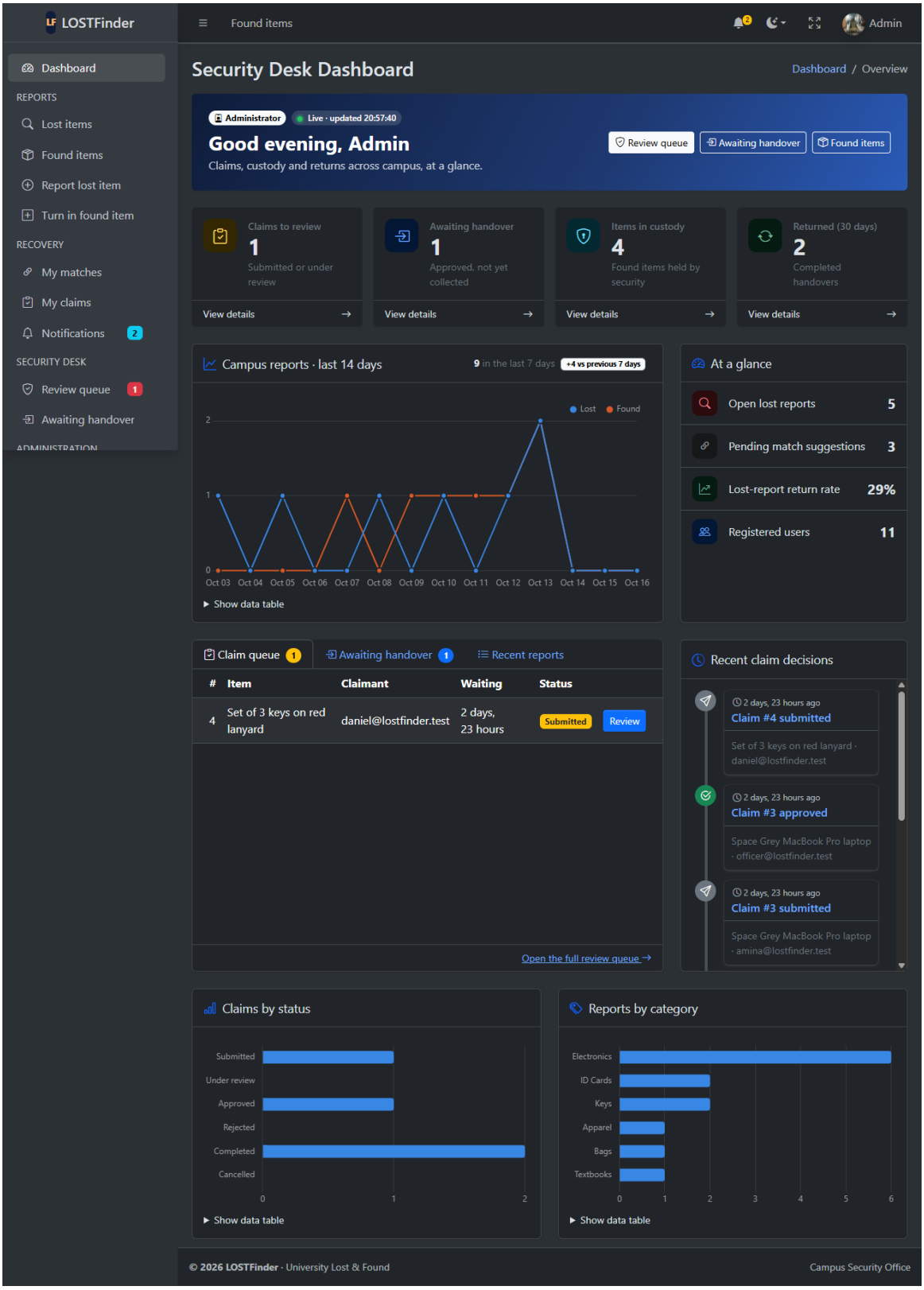
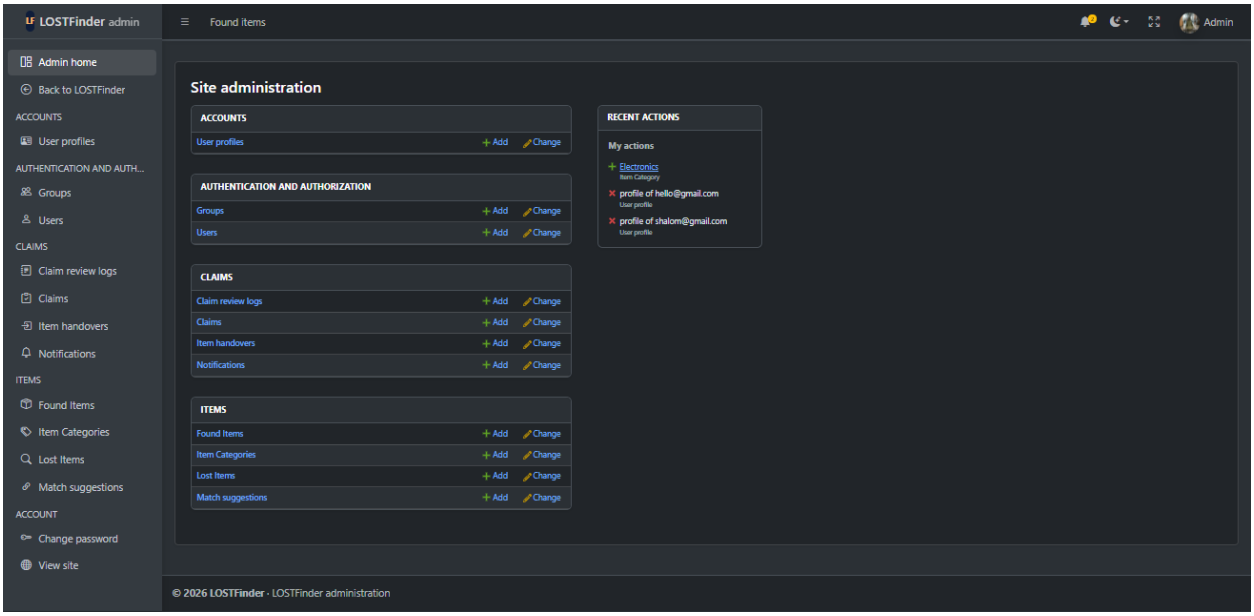


***Figure 5.10:*** *Django admin dashboard for an administrator*




# CHAPTER 6: CONCLUSION AND RECOMMENDATIONS

## 6.1 Conclusion

LOSTFinder meets the six objectives set in section 1.2. It replaces word of mouth with one system for reporting, matching and returning lost items, and keeps a full record of every claim and handover.

| Objective | Result |
|---|---|
| 1. Report lost and found items | Done: full CRUD with photos, search and filters |
| 2. Suggest matches automatically | Done: scored out of 100 on every save; pairs of 35 or more suggested |
| 3. Verify claims and record handovers | Done: six-state workflow with an audit log and notifications |
| 4. Dashboards for each role | Done: AdminLTE dashboards for members and for the security desk, refreshed every 30 seconds |
| 5. Restrict pages by sign-in and role | Done: sign-in required everywhere; review pages limited to reviewers |
| 6. Meet the course brief | Done: 4 apps, 9 related models, validated forms, AdminLTE, Django admin, 158 passing tests |

## 6.2 Limitations

- Notifications appear in the app only, with no email or SMS.
- Matching compares words and characters only. It does not recognise synonyms such as "phone" and "mobile", and it does not compare photos.
- The app is set up for local development. It has not been configured for a public server.

## 6.3 Recommendations

1. Send email notifications through Django's email framework.
2. Improve matching with a list of synonyms per category, and later with image similarity.
3. Prepare for deployment: set `ALLOWED_HOSTS`, run `collectstatic`, use PostgreSQL, and serve the app with a production web server.
4. Test on Python 3.12 and 3.13, and lower the minimum Python version if the tests pass, so the app is easier to install.

# REFERENCES


1. Kumar, K. *Django Web Development Midterm Examination, Project Work: Design and Development of a Database-Driven Web Application Using Django.* Web Technologies and API Services course brief, AUCA.
2. Django Software Foundation. *Django documentation.* docs.djangoproject.com/en/stable
3. Django Software Foundation. *Getting started with Django.* djangoproject.com/start
4. ColorlibHQ. *AdminLTE documentation.* adminlte.io/docs
5. Bootstrap team. *Bootstrap 5.3 documentation.* getbootstrap.com/docs/5.3
6. Bootstrap team. *Bootstrap Icons.* icons.getbootstrap.com
7. Chart.js contributors. *Chart.js documentation.* chartjs.org/docs/latest
8. Python Software Foundation. *difflib: Helpers for computing deltas.* docs.python.org/3/library/difflib.html
9. Pillow contributors. *Pillow documentation.* pillow.readthedocs.io
10. Astral. *uv documentation.* docs.astral.sh/uv


# APPENDIX A: SUBMISSION CHECKLIST

The brief asks for five deliverables. Each one is listed below with where to find it.

- Complete Django source code: github.com/shaloomniyibizi/LOSTFinder
- Migration files: each app's `migrations/` folder
- Database file: `db.sqlite3` and the `media/` folder are in the zip file, because `.gitignore` excludes them from GitHub.
- Login credentials: section 4.6
- ER diagram with cardinality: sections 3.2 and 3.3
- Screenshots of the dashboard and major functions: section 5.5
- Project report: this document

# APPENDIX B: SUGGESTED DEMONSTRATION SEQUENCE

This order follows the marking scheme, so each criterion is shown once and in full. Before starting, run the server and load the demo data (section 4.6).

| Step | What to show | Marking criterion (marks) |
|---|---|---|
| 1 | Open the project in Visual Studio Code. Show `config/settings.py`: the four apps, templates, static and media folders, and the `.env` settings. | Project structure and configuration (5) |
| 2 | Show the four app folders, each with its own `models.py`, `views.py` and `urls.py`. | At least two Django apps (2) |
| 3 | Trace `/items/lost/5/` from `config/urls.py` to `items/urls.py` to `lost_item_detail`, then open the | URL routing and views (2) |

| Step | What to show | Marking criterion (marks) |
|---|---|---|
| | page (section 3.1). | |
| 4 | Show the blocks in base.html, then {% extends "base.html" %} in items/item_list.html. | Templates and template inheritance (2) |
| 5 | Show the models and the ER diagram, and explain one relationship of each type. Show the migration files. | Database models and relationships (3) |
| 6 | Sign in as amina@lostfinder.test. Show the sidebar, top bar, summary cards, charts and the table of reports. | AdminLTE integration and dashboard (3) |
| 7 | Report a lost item with a future date to show the error, then fix and save it. Edit it, then delete it through the confirmation page. | CRUD operations and validation (4) |
| 8 | Open **My matches** and accept a match. Claim a found item, sign in as officer@lostfinder.test, approve the claim and record the handover. | CRUD operations and validation (4) |
| 9 | Sign in at /admin/ as the superuser and edit a record. Change a user's role to security officer. As a student, open /claims/review/ to show the 403 page, then sign out. | Django admin and authentication (4) |
| 10 | Each group member explains one area: URLs and views, models, forms and validation, templates and AdminLTE, matching, or the claim workflow. | Demonstration and individual understanding (3) |